

6장 합성곱 신경망 2 (6.1.5~)

6.1 이미지 분류를 위한 신경망

ResNet

깊어진 신경망을 효과적으로 학습하기 위한 방법

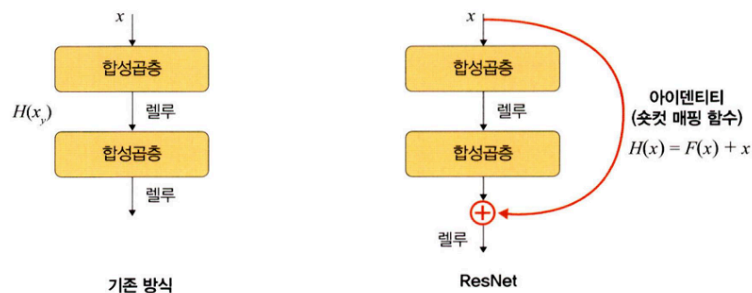
residual 개념을 고안한 것

신경망은 깊이가 깊어질수록 성능이 좋아지다가 일정 단계에 다다르면 오히려 성능이 나빠짐

네트워크 깊이가 깊다고 해서 무조건 성능이 좋아지지 않음

ResNet은 이런 문제를 해결하기 위해 residual block을 도입

residual block은 기울기가 잘 전파될 수 있도록 일종의 short cut을 만들어줌



블록은 계층의 묶음

합성곱층을 하나의 블록으로 묶은 것

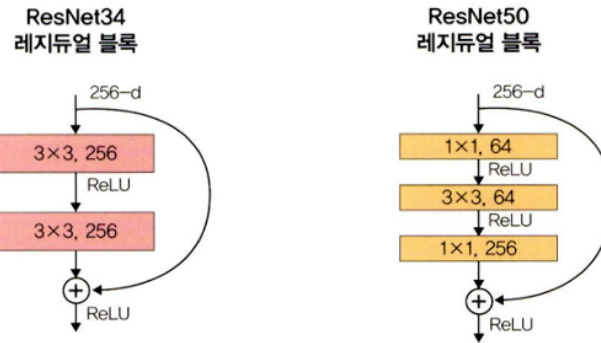
이렇게 묶인 계층들이 하나의 residual block이 됨

그런 residual block을 여러 개 쌓은 것이 ResNet

이렇게 계층을 계속해서 쌓아 늘리면 파라미터 수가 문제가 됨

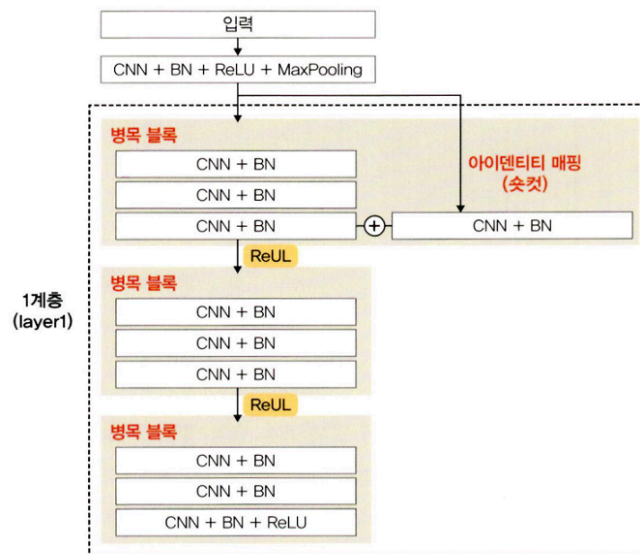
계층이 깊어질수록 파라미터는 무제한으로 커짐

⇒ 병목 블록 (bottleneck block)을 둬



ResNet34는 기본 블록, ResNet50은 병목 블록을 사용한 경우
 기본 블록에서는 파라미터 수가 39.3216M이지만 병목 블록에서는 파라미터 수가 6.9632M
 깊이가 깊어졌음에도 파라미터 수는 감소함
 병목 블록을 사용하면 파라미터 수가 감소하는 효과를 줌
 병목 블록을 사용한 ResNet50에서는 3×3 합성곱층 앞뒤로 1×1 합성곱층이 붙어 있음
 1×1 합성곱층의 채널 수를 조절하면서 차원을 줄였다 늘리는 것이 가능하여 파라미터 수를 줄일 수 있음

아이덴티티 매핑



그림에서 +는 아이덴티티 매핑을 의미함
 입력 x 가 어떤 함수를 통과하더라도 다시 x 라는 형태로 출력되게 함

```
def forward(self, x):
    i = x
```

```

x = self.conv1(x)
x = self.bn1(x)
x = self.relu(x)
x = self.conv2(x)
x = self.bn2(x)

if self.downsample is not None:
    i = self.downsample(i)

x += i #아이덴티티 매핑 적용
x = self.relu(x)
return x

```

코드를 보면 입력 x를 i라는 변수에 저장함

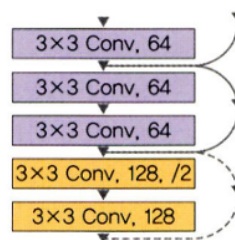
입력 x는 합성곱층을 통과하다가 마지막 x에 I를 더해줌

합성곱층을 통과하면서 같은 형태를 더하기 때문에 입력 형태와 최종 형태가 동일함

다운샘플

특성 맵 크기를 줄이기 위한 것

풀링과 같은 역할



ResNet 일부

보라색 영역의 첫번째 블록에서 특성 맵의 형상이 (28, 28, 64)이면

세 번째 블록의 마지막 합성곱층을 통과하고 아이덴티티 매핑까지 완료된 특성 맵의 형상도 (28, 28, 64)

노란색 영역의 시작 지점에서는 채널 수가 128로 늘어나고 첫 번째 블록에서 합성곱층의 스트라이드가 2로 늘어나 (14, 14, 128)로 바뀜

→ 보라색과 노란색의 형태가 달라 아이덴티티 매핑을 할 수가 없음

⇒ 아이덴티티에 대해 다운 샘플이 필요

입력과 출력의 형태를 맞추기 위해서는 스트라이드 2를 가진 1x1 합성곱 계층을 하나 연결해야함

이처럼 입력과 출력의 차원이 같은 것을 아이덴티티 블록이라고 함

입력과 출력의 차원이 동일하지 않고 변경해야 하는 것을 프로젝션 숏컷, 합성곱 블록이라고 함

ResNet 구현

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
import torch.utils.data as data
import torchvision
import torchvision.transforms as transforms
import torchvision.datasets as datasets
import torchvision.models as models

import matplotlib.pyplot as plt
import numpy as np

import copy
from collections import namedtuple
import os
import random
import time

import cv2
from torch.utils.data import DataLoader, Dataset
from PIL import Image

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
```

```
!unzip -nq /content/sample_data/chap05/dogs-vs-cats.zip -d /content/sample_data/chap05/
```

```
#6-66 훈련과 테스트 데이터셋 불러오기
cat_directory = r'/content/sample_data/chap05/dogs-vs-cats/cat/'
dog_directory = r'/content/sample_data/chap05/dogs-vs-cats/dog/'

cat_images_filepaths = sorted([os.path.join(cat_directory, f) for f in
                                os.listdir(cat_directory)])
dog_images_filepaths = sorted([os.path.join(dog_directory, f) for f in
                                os.listdir(dog_directory)])
images_filepaths = [*cat_images_filepaths, *dog_images_filepaths]
correct_images_filepaths = [i for i in images_filepaths if cv2.imread(i) is not None]

#6-69 데이터셋을 훈련, 검증, 테스트 용도로 분리
random.seed(42)
random.shuffle(correct_images_filepaths)
train_images_filepaths = correct_images_filepaths[:400]
val_images_filepaths = correct_images_filepaths[400:10]
test_images_filepaths = correct_images_filepaths[10:]
print(len(train_images_filepaths), len(val_images_filepaths), len(test_images_filepaths))

400 92 10
```

```
#6-70 이미지에 대한 레이블 구분
class DogsCatDataset(Dataset):
    def __init__(self, file_list, transform=None, phase='train'):
        self.file_list = file_list
        self.transform = transform
        self.phase = phase

    def __len__(self):
        return len(self.file_list)

    def __getitem__(self, idx):
        img_path = self.file_list[idx]
        img = image.open(img_path)
        img_transformed = self.transform(img, self.phase)

        label = img_path.split('/')[-1].split('.')[0]
        if label == 'dog':
            label = 1
        elif label == 'cat':
            label = 0
        return img_transformed, label
```

```
#6-66 이미지 데이터 전처리
class ImageTransform():
    def __init__(self, resize, mean, std):
        self.data_transform = {
            'train': transforms.Compose([
                transforms.RandomResizedCrop(resize, scale = (0.5, 1.0)),
                transforms.RandomHorizontalFlip(),
                transforms.ToTensor(),
                transforms.Normalize(mean, std)
            ]),
            'val': transforms.Compose([
                transforms.Resize(256),
                transforms.CenterCrop(resize),
                transforms.ToTensor(),
                transforms.Normalize(mean, std)
            ])
        }

    def __call__(self, img, phase):
        return self.data_transform[phase](img)
```

```
#6-67 변수에 대한 값 정의
size = 224
mean = (0.485, 0.456, 0.406)
std = (0.229, 0.224, 0.225)
batch_size = 32
```

```
#6-71 데이터 데이터셋 정리
train_dataset = DogsCatDataset(train_images_filepaths, transform=ImageTransform(size, mean, std), phase='train')
val_dataset = DogsCatDataset(val_images_filepaths, transform=ImageTransform(size, mean, std), phase='val')

index = 9
print(train_dataset.__getitem__(index)[0].size())
print(train_dataset.__getitem__(index)[1])

0

torch.Size([3, 224, 224])

#6-72 데이터셋의 데이터를 묶음으로 불러오기
train_iterator = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
valid_iterator = DataLoader(val_dataset, batch_size=batch_size, shuffle=False)
data_loader_dict = {'train': train_iterator, 'val': valid_iterator}

batch_iterator = iter(train_iterator)
inputs, label = next(batch_iterator)
print(inputs.size())
print(label)

torch.Size([32, 3, 224, 224])
tensor([0, 1, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 1, 0, 1, 0, 1, 1, 1, 1, 0, 1, 0, 1, 0, 1, 1])
```

위 코드들은 이전 6장에서 진행했던 VGGNet, AlexNet 등과 같은 구조

라이브러리 - 이미지 데이터 전처리 - 변수 값 정의 - 훈련, 테스트 데이터셋 불러오기 - 레이블 구분

```
#6-73 기본 블록 정의
class BasicBlock(nn.Module):
    expansion = 1

    def __init__(self, in_channels, out_channels, stride=1, downsample=False):
        super().__init__()
        self.conv1 = nn.Conv2d(in_channels, out_channels, kernel_size=3,
                                stride=stride, padding=1, bias=False)
        self.bn1 = nn.BatchNorm2d(out_channels)
        self.conv2 = nn.Conv2d(out_channels, out_channels, kernel_size=3,
                                stride=1, padding=1, bias=False)
        self.bn2 = nn.BatchNorm2d(out_channels)
        self.relu = nn.ReLU(inplace=True)

        if downsample:
            conv = nn.Conv2d(in_channels, out_channels, kernel_size=1,
                              stride=stride, bias=False)
            bn = nn.BatchNorm2d(out_channels)
            downsample = nn.Sequential(conv, bn)
        else:
            downsample = None
        self.downsample = downsample

    def forward(self, x):
        l = x
        x = self.conv1(x)
        x = self.bn1(x)
        x = self.relu(x)
        x = self.conv2(x)
        x = self.bn2(x)

        if self.downsample is not None:
            l = self.downsample(l)

        x += l
        x = self.relu(x)

        return x
```

```
#6-74 병목 블록 정의
class Bottleneck(nn.Module):
    expansion = 4

    def __init__(self, in_channels, out_channels, stride=1, downsample=False):
        super().__init__()
        self.conv1 = nn.Conv2d(in_channels, out_channels, kernel_size=1,
                                stride=1, bias=False)
        self.bn1 = nn.BatchNorm2d(out_channels)
        self.conv2 = nn.Conv2d(out_channels, out_channels, kernel_size=3,
                                stride=stride, padding=1, bias=False)
        self.bn2 = nn.BatchNorm2d(out_channels)
        self.conv3 = nn.Conv2d(out_channels, self.expansion*out_channels,
                                kernel_size=1, stride=1, bias=False)
        self.bn3 = nn.BatchNorm2d(self.expansion*out_channels)
        self.relu = nn.ReLU(inplace=True)

        if downsample:
            conv = nn.Conv2d(in_channels, self.expansion*out_channels, kernel_size=1,
                              stride=stride, bias=False)
            bn = nn.BatchNorm2d(self.expansion*out_channels)
            downsample = nn.Sequential(conv, bn)
        else:
            downsample = None
        self.downsample = downsample

    def forward(self, x):
        l = x
        x = self.conv1(x)
        x = self.bn1(x)
        x = self.relu(x)
        x = self.conv2(x)
        x = self.bn2(x)
        x = self.relu(x)
        x = self.conv3(x)
        x = self.bn3(x)

        if self.downsample is not None:
            l = self.downsample(l)

        x += l
        x = self.relu(x)
        return x
```

ResNet의 전체 네트워크 구성을 위한 기본 블록과 병목 블록에 대한 정의

기본 블록은 ResNet18, ResNet34에서 사용되며 합성곱 두 개로 구성됨

입력 데이터의 크기와 네트워크를 통과한 후 데이터 크기가 다를 경우를 위해 합성곱층에 스트라이드를 적용해서 다운 샘플을 함

forward()에서 아이덴티티 매핑을 적용

특정 층에 존재하는 출력 결과를 다음 합성곱층을 통과한 출력 결과에 더해줌

병목 블록은 ResNet50, 101, 152에서 사용되며 1x1 합성곱층, 3x3 합성곱층, 1x1 합성곱층으로 구성됨

계층을 더 깊게 쌓으면서 계산에 대한 비용을 줄일 수 있음

계층이 많아진다는 것은 활성화 함수가 기존보다 더 많이 포함되어 더 많은 비선형성을 처리할 수 있음을 의미함 → 다양한 입력 데이터에 대한 처리가 가능

아이덴티티 매핑과 병목 블록을 활용하여 ResNet 네트워크에 더욱 깊은 계층을 쌓을 수 있게 됨

```
#6-76 ResNet 모델 네트워크
class ResNet(nn.Module):
    def __init__(self, config, output_dim, zero_init_residual=False):
        super().__init__()

        block, n_blocks, channels = config
        self.in_channels = channels[0]
        assert len(n_blocks) == len(channels) == 4
        self.conv1 = nn.Conv2d(3, self.in_channels, kernel_size=7, stride=2,
                                padding=3, bias=False)
        self.bn1 = nn.BatchNorm2d(self.in_channels)
        self.relu = nn.ReLU(inplace=True)
        self.maxpool = nn.MaxPool2d(kernel_size=3, stride=2, padding=1)

        self.layer1 = self.get_resnet_layer(block, n_blocks[0], channels[0])
        self.layer2 = self.get_resnet_layer(block, n_blocks[1], channels[1], stride=2)
        self.layer3 = self.get_resnet_layer(block, n_blocks[2], channels[2], stride=2)
        self.layer4 = self.get_resnet_layer(block, n_blocks[3], channels[3], stride=2)

        self.avgpool = nn.AdaptiveAvgPool2d((1, 1))
        self.fc = nn.Linear(self.in_channels, output_dim)

        if zero_init_residual:
            for m in self.modules():
                if isinstance(m, Bottleneck):
                    nn.init.constant_(m.bn3.weight, 0)
                elif isinstance(m, BasicBlock):
                    nn.init.constant_(m.bn2.weight, 0)

    def get_resnet_layer(self, block, n_blocks, channels, stride=1):
        layers = []
        if self.in_channels != block.expansion * channels:
            downsample = True
        else:
            downsample = False

        layers.append(block(self.in_channels, channels, stride, downsample))
        for i in range(1, n_blocks):
            layers.append(block(block.expansion*channels, channels))
```

```
self.in_channels = block.expansion * channels
return nn.Sequential(*layers)

def forward(self, x):
    x = self.conv1(x)
    x = self.bn1(x)
    x = self.relu(x)
    x = self.maxpool(x)
    x = self.layer1(x)
    x = self.layer2(x)
    x = self.layer3(x)
    x = self.layer4(x)
    x = self.avgpool(x)
    h = x.view(x.shape[0], -1)
    x = self.fc(h)
    return x, h
```

ResNet 모델에 대한 네트워크 정의

합성곱층과 잔차 블록을 반복해 이미지의 특징을 추출하고 평균 풀링과 완전연결층을 통해 클래스를 분류

```
#6-76 ResNetConfig 정의
ResNetConfig = namedtuple('ResNetConfig', ['block', 'n_blocks', 'channels'])

#6-77 기본 블록을 사용하여 ResNetConfig 정의
resnet18_config = ResNetConfig(block=BasicBlock,
                                n_blocks=[2, 2, 2, 2],
                                channels=[64, 128, 256, 512])

resnet32_config = ResNetConfig(block=BasicBlock,
                                n_blocks=[3, 4, 6, 3],
                                channels=[64, 128, 256, 512])

#6-78 병목 블록을 사용하여 ResNetConfig 정의
resnet50_config = ResNetConfig(block=Bottleneck,
                                n_blocks=[3, 4, 6, 3],
                                channels=[64, 128, 256, 512])

resnet101_config = ResNetConfig(block=Bottleneck,
                                n_blocks=[3, 4, 23, 3],
                                channels=[64, 128, 256, 512])

resnet152_config = ResNetConfig(block=Bottleneck,
                                n_blocks=[3, 8, 36, 3],
                                channels=[64, 128, 256, 512])
```

```
#6-79 사전 훈련된 ResNet 모델 사용
pretrained_model = models.resnet50(pretrained=True)

/usr/local/lib/python3.13/dist-packages/torchvision/models/_utils.py:208: UserWarning: The
warnings.warn(
/usr/local/lib/python3.13/dist-packages/torchvision/models/_utils.py:223: UserWarning: Argument
warnings.warn(msg)

#6-80 사전 훈련된 ResNet 네트워크 확인
print(pretrained_model)

숨겨진 출력 표시

#6-81 ResNet50 Config를 사용한 ResNet 모델 사용
OUTPUT_DIM = 2
model = ResNet(resnet50_config, OUTPUT_DIM)
print(model)

숨겨진 출력 표시

#6-82 옵티마이저와 손실 함수 정의
optimizer = optim.Adam(model.parameters(), lr=1e-7)
criterion = nn.CrossEntropyLoss()

model = model.to(device)
criterion = criterion.to(device)

#6-83 모델 학습 정확도 측정 함수 정의
def calculate_topk_accuracy(y_pred, y, k=2):
    with torch.no_grad():
        batch_size = y.shape[0]
        _, top_pred = y_pred.topk(k, 1)
        top_pred = top_pred.t()
        correct = top_pred.eq(y.view(1, -1).expand_as(top_pred))
        correct_1 = correct[:1].reshape(-1).float().sum(0, keepdim=True)
        correct_k = correct[:k].reshape(-1).float().sum(0, keepdim=True)
        acc_1 = correct_1 / batch_size
        acc_k = correct_k / batch_size
    return acc_1, acc_k
```

기본 블록을 사용하는 ResNet18과 32의 config를 정의
병목 블록을 사용하는 ResNet50, 101, 152의 config 정의

ResNet50 모델을 사용, 두 개의 클래스 (개와 고양이)이므로 OUTPUT_DIM은 2
앞서 직접 정의한 ResNet 네트워크와 사전 훈련된 ResNet 네트워크 차이점이 없음
옵티마이저는 Adam, 손실함수는 CrossEntropyLoss 사용
모델이 얼마나 잘 학습되었는지 측정하기 위한 함수를 정의

```
#6-84 모델 학습 함수 정의
def train(model, iterator, optimizer, criterion, device):
    epoch_loss = 0
    epoch_acc_1 = 0
    epoch_acc_5 = 0

    model.train()
    for (x, y) in iterator:
        x = x.to(device)
        y = y.to(device)

        optimizer.zero_grad()
        y_pred = model(x)
        loss = criterion(y_pred[0], y)

        acc_1, acc_5 = calculate_topk_accuracy(y_pred[0], y)
        loss.backward()
        optimizer.step()

        epoch_loss += loss.item()
        epoch_acc_1 += acc_1.item()
        epoch_acc_5 += acc_5.item()

    epoch_loss /= len(iterator)
    epoch_acc_1 /= len(iterator)
    epoch_acc_5 /= len(iterator)
    return epoch_loss, epoch_acc_1, epoch_acc_5
```

```
#6-85 모델 평가 함수 정의
def evaluate(model, iterator, criterion, device):
    epoch_loss = 0
    epoch_acc_1 = 0
    epoch_acc_5 = 0

    model.eval()
    for (x, y) in iterator:
        x = x.to(device)
        y = y.to(device)
        y_pred = model(x)
        loss = criterion(y_pred[0], y)

        acc_1, acc_5 = calculate_topk_accuracy(y_pred[0], y)
        epoch_loss += loss.item()
        epoch_acc_1 += acc_1.item()
        epoch_acc_5 += acc_5.item()

    epoch_loss /= len(iterator)
    epoch_acc_1 /= len(iterator)
    epoch_acc_5 /= len(iterator)
    return epoch_loss, epoch_acc_1, epoch_acc_5

#6-86 모델 학습 시간 측정 함수 정의
def epoch_time(start_time, end_time):
    elapsed_time = end_time - start_time
    elapsed_mins = int(elapsed_time / 60)
    elapsed_secs = int(elapsed_time - (elapsed_mins * 60))
    return elapsed_mins, elapsed_secs
```

모델 학습 함수와 모델 평가 함수에 대해서 정의

모델을 학습시키는데 어느 정도 시간이 걸리는지 측정하기 위한 함수를 정의

```
#6-87 모델 학습
def train(model, iterator, optimizer, criterion, device):
    epoch_loss = 0
    epoch_acc_1 = 0
    epoch_acc_5 = 0

    model.train()
    for (x, y) in iterator:
        x = x.to(device)
        y = y.to(device)

        optimizer.zero_grad()
        y_pred = model(x)
        loss = criterion(y_pred[0], y)

        acc_1, acc_5 = calculate_topk_accuracy(y_pred[0], y)
        loss.backward()
        optimizer.step()

        epoch_loss += loss.item()
        epoch_acc_1 += acc_1.item()
        epoch_acc_5 += acc_5.item()

    epoch_loss /= len(iterator)
    epoch_acc_1 /= len(iterator)
    epoch_acc_5 /= len(iterator)
    return epoch_loss, epoch_acc_1, epoch_acc_5
```

```
#6-88 테스트 데이터셋을 이용한 모델 예측
import pandas as pd
id_list = []
pred_list = []

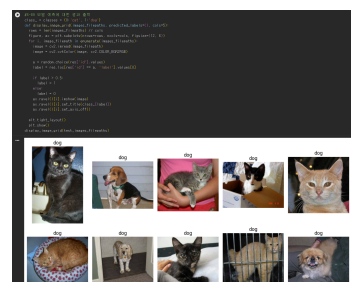
with torch.no_grad():
    for test_path in test_images_filenames:
        img = image.open(test_path)
        id = test_path.split('/')[-1].split('.')[0]
        transform = image.transforms.Compose([
            image.transforms.Resize(224),
            image.transforms.ToTensor(),
            image.transforms.Normalize([0.485, 0.456, 0.422], [0.229, 0.224, 0.225])
        ])
        img = transform(img)
        img = img.unsqueeze(0)
        img = img.to(device)

        model.eval()
        outputs = model(img)
        preds = F.softmax(outputs[0], dim=-1).tolist()
        id_list.append(id)
        pred_list.append(preds[0])

res = pd.DataFrame({
    'id': id_list,
    'label': pred_list
})

res.sort_values(by='id', inplace=True)
res.reset_index(drop=True, inplace=True)

res.to_csv('content/sample_data/chapter06/ResNet_model.csv', index=False)
res.head(10)
```



훈련과 검증 데이터셋을 이용하여 모델을 학습 시킴

모델 학습 결과에 대한 오차와 정확도 측면에서 성능이 좋지 못함

이미지 데이터의 개수를 늘린다면 성능이 좋아질 수도 있음 (현재는 빠른 학습을 위해 데이터 개수를 제한시켜놓음)

테스트 데이터셋의 예측 결과인 ResNet.csv를 출력
레이블이 0.5보다 크면 개, 작으면 고양이를 의미함

모델이 얼마나 잘 예측했는지 이미지를 통해 확인

테스트 데이터셋을 이용한 예측 결과를 이미지로 출력

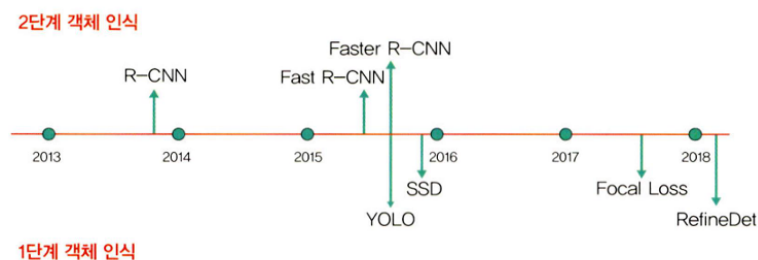
정확도가 높지 않음을 알 수 있음

6.2 객체 인식을 위한 신경망

객체 인식은 이미지나 영상 내에 있는 객체를 식별하는 컴퓨터 비전 기술

이미지나 영상 내에 있는 여러 객체에 대해 각 객체가 무엇인지 분류하는 문제와 그 객체 위치가 어디인지 박스로 나타내는 위치 검출 문제를 다루는 분야

딥러닝을 이용한 객체 인식 알고리즘은 크게 1단계와 2단계 객체 인식으로 나눌 수 있음



1단계 객체 인식은 분류와 위치 검출 두 문제를 동시에 행하는 방법

2단계 객체 인식은 이 두 문제를 순차적으로 행하는 방법

1단계는 비교적 빠르지만 정확도가 낮음

2단계는 비교적 느리지만 정확도가 높음

1단계 객체 인식에는 YOLO 계열과 SSD 계열 등을 포함

2단계 객체 인식에는 R-CNN 계열이 대표적

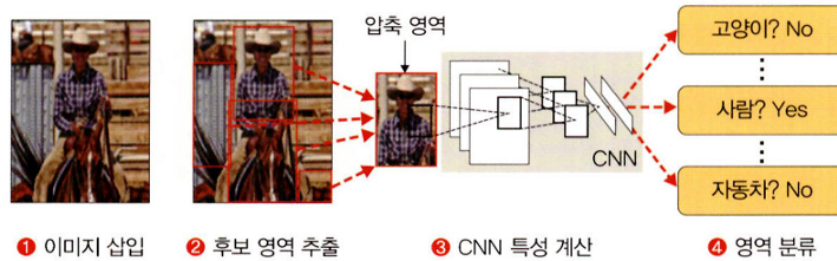
R-CNN

이전 객체 인식 알고리즘들은 일정한 크기를 가지는 윈도우를 가지고 이미지의 모든 영역을 탐색하면서 객체를 검출해 내는 슬라이딩 윈도우 방식을 사용

알고리즘의 비효율성 때문에 현재는 선택적 탐색 알고리즘을 적용한 후보 영역 방식을 주로 사용

후보 영역은 영상, 이미지의 모든 구간을 탐색하는 것이 아닌 객체가 있을 법한 영역만을 탐색

R-CNN은 이미지 분류를 수행하는 CNN과 이미지에서 객체가 있을만한 영역을 제안해주는 후보 영역 알고리즘을 결합한 알고리즘



이미지 입력을 받아 2000개의 바운딩 박스를 선택적 탐색 알고리즘으로 추출후 잘라냄
 CNN 모델에 넣기 위해 같은 크기로 통일
 각각 분류를 진행하여 결과를 도출

R-CNN은 성능이 뛰어나지만 단점이 존재함

- 복잡한 학습 과정, 긴 학습 시간과 대용량 저장 공간, 객체 검출 속도 문제

⇒ Fast R-CNN

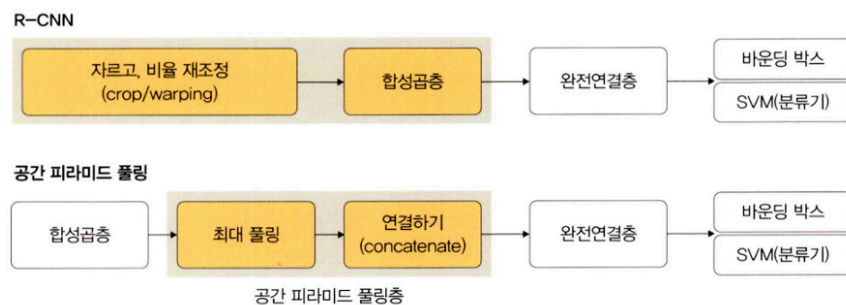
공간 피라미드 풀링

기존 CNN 구조들은 모두 완전 연결층을 위해 입력 이미지를 고정해야했음

이미지를 고정된 크기로 자르거나 비율을 조정함

→ 물체의 일부분이 잘리거나 본래의 생김새와 달라지게 됨

⇒ 공간 피라미드 풀링



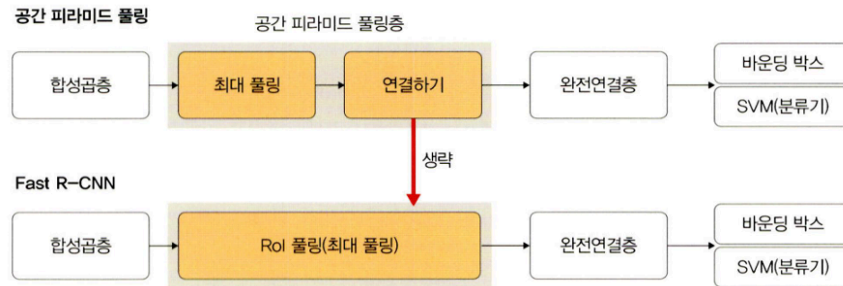
입력 이미지의 크기와 상관없이 합성곱층을 통과시키고 완전연결층에 전달되기 전에 특성 맵들을 동일한 크기로 조절해주는 풀링층을 적용하는 기법

원본 이미지의 특징이 훼손되지 않는 특성 맵을 얻을 수 있음

Fast R-CNN

R-CNN은 바운딩 박스마다 CNN을 돌려서 긴 학습 시간을 가짐

⇒ Fast R-CNN은 속도 문제를 개선하기 위해 RoI 풀링을 도입



선택적 탐색에서 찾은 바운딩 박스 정보가 CNN을 통과하면서 유지되도록 하고 최종 CNN 특성 맵은 풀링을 적용하여 완전 연결층을 통과하도록 크기를 조정
→ 바운딩 박스마다 CNN을 돌리는 시간을 단축할 수 있음

Faster R-CNN

더욱 빠른 객체 인식을 수행하기 위한 네트워크

Fast R-CNN에서 문제였던 후보 영역 생성을 CNN 내부 네트워크에서 진행할 수 있도록 설계

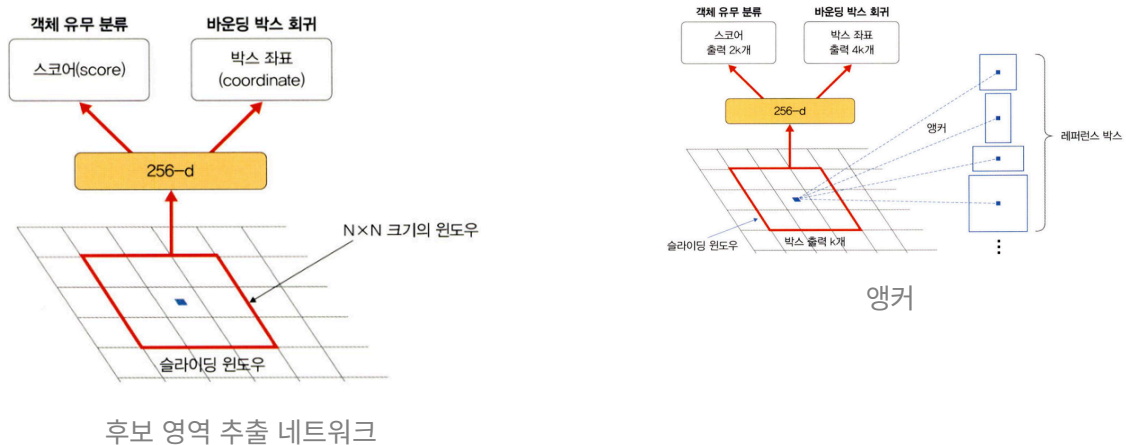
Fast R-CNN에 후보 영역 추출 네트워크를 추가한 것 → 외부의 느린 선택적 탐색 대신 내부의 빠른 RPN을 사용

후보 영역 추출 네트워크는 특성 맵 $N \times N$ 크기의 작은 윈도우 영역을 입력으로 받음

→ 해당 영역에 객체의 존재 유무 판단을 위해 이진 분류를 수행하는 작은 네트워크를 생성

→ 위치 보정을 위해 바운딩 박스 회귀 추가

→ 하나의 특성 맵에서 모든 영역에 대한 객체의 존재 유무를 확인하기 위해 슬라이딩 윈도우 방식으로 설계한 작은 윈도우 영역을 이용해 객체 탐색



이미지에 존재하는 객체들의 크기와 비율이 다양해 고정된 크기의 입력만으로는 모두 수용하기 어려움

⇒ 앵커 방식

여러 크기와 비율의 레퍼런스 박스 k 개를 미리 정의하고 각각의 슬라이딩 윈도우 위치마다

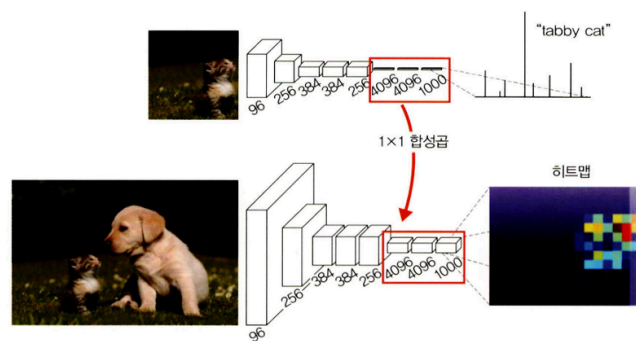
박스 k개를 출력하도록 설계

특성 맵 크기가 $w \times h$ 이면 하나의 특성 맵에 앵커가 총 $w \times h \times k$ 개 존재

6.3 이미지 분할을 위한 신경망

신경망을 훈련시켜 이미지를 픽셀 단위로 분할하여 이미지에 포함된 객체를 추출

완전 합성곱 네트워크



고정된 크기의 입력만 받아들이고 거친 후에는 위치 정보가 사라지는 완전연결층의 한계를 극복하기 위해 완전 연결층을 1x1 합성곱으로 대체

이미지 분류에서 우수한 성능을 보인 CNN 기반 모델을 변형시켜 이미지 분할에 적합하도록 만든 네트워크

합성곱층으로 사용되어서 입력 이미지에 대한 크기 제약이 사라지기도 함

합성곱 & 역합성곱 네트워크

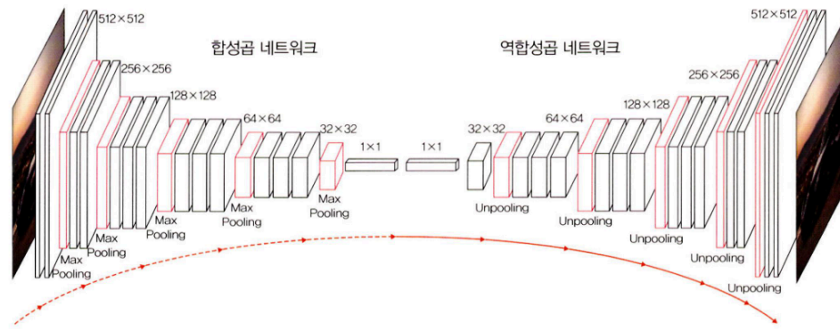
완전 합성곱 네트워크는 위치 정보가 보존되지만 단점도 존재

- 여러 단계의 합성곱층과 풀링층을 거치면서 해상도가 낮아짐
 - 낮아진 해상도를 복원하기 위해 업 샘플링 방식을 사용하기 때문에 이미지의 세부 정보들을 잃어버리는 문제가 발생
- ⇒ 역합성곱 네트워크 도입

CNN의 최종 출력 결과를 원래의 입력 이미지와 같은 크기로 만들고 싶을 때 역합성곱을 사용 (= 업 샘플링)

특성 맵 크기를 증가시키는 방식으로 동작

- 각각의 픽셀 주위로 제로 패딩을 추가 → 패딩된 것에 합성곱 연산을 수행



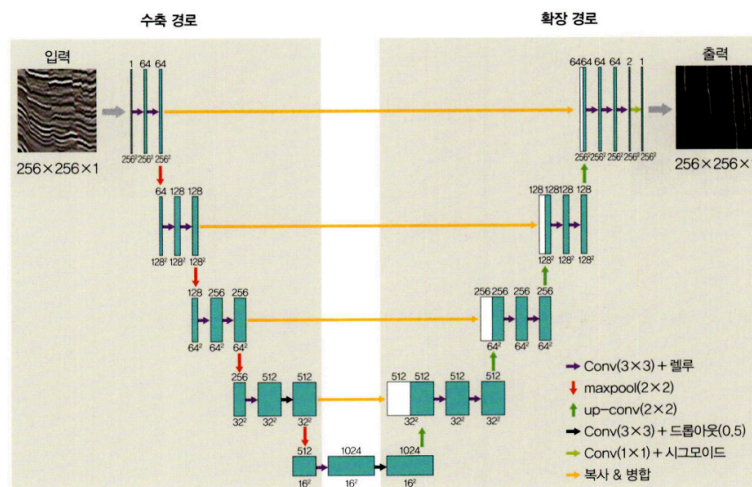
U-Net

바이오 메디컬 이미지 분할을 위한 합성곱 신경망

이전 패치에서 검증이 끝난 부분을 다음 패치에서 또 검증하는 기존 슬라이딩 윈도우 방식은 속도가 느렸지만 U-Net에서는 이미 검증이 끝난 패치는 건너뛰어서 속도가 빠름

일반적으로 패치 크기가 커진다면 넓은 범위를 인식해 컨텍스트 인식에 탁월하지만 지역화에 한계가 있음

U-Net에서는 컨텍스트 인식과 지역화의 트레이드오프 문제를 개선함



U-Net은 FCN 기반으로 수축 경로와 확장 경로로 구성됨

수축 경로는 컨텍스트를 포착

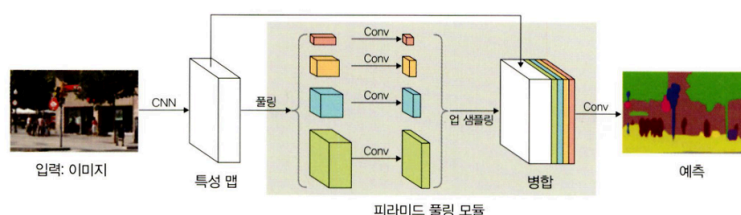
확장 경로는 특성 맵을 업 샘플링하고 수축 경로에서 포착한 특성 맵의 컨텍스트와 결합하여 정확한 지역화를 수행

3x3 합성곱이 주를 이루고 각 합성곱 블록은 3x3 합성곱 두개로 구성되고 그 사이에 드롭아웃이 있음

그림에서는 왼쪽 수축 경로에서의 블록은 3x3 합성곱 두개로 구성된 것이 4개가 있는 형태 각 블록은 최대 풀링을 이용하여 크기를 줄이면서 다음 블록으로 넘어감

오른쪽 확장 경로에서는 합성곱 블록에 up-conv을 붙임
 수축 과정에서 줄어든 크기를 다시 키워 가면서 합성곱 블록을 이용하는 형태
 크기가 다양한 이미지의 객체를 분할하기 위해 크기가 다양한 특성 맵을 병합할 수 있도록
 다운 샘플링과 업 샘플링을 순서대로 반복하는 구조

PSPNet



시멘틱 분할 알고리즘

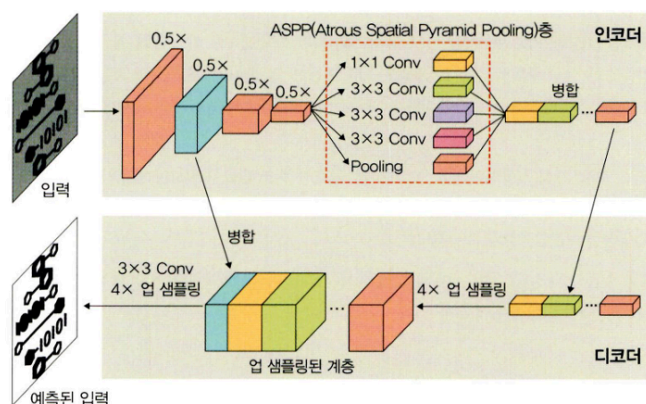
완전연결층의 한계를 극복하기 위해 피라미트 풀링 모듈을 추가

이미지 출력이 서로 다른 크기가 되도록 여러 차례 풀링을 진행 (각각 다른 크기의 특성 맵은 서로 다른 영역들의 정보를 담음)

- 1x1 합성곱을 이용해 채널 수를 조정
- 모듈의 입력 크기에 맞게 특성 맵을 업 샘플링함 (양선형 보간법 사용)
- 원래의 특성 맵과 앞선 과정에서 새로 생성된 특성 맵들을 병합

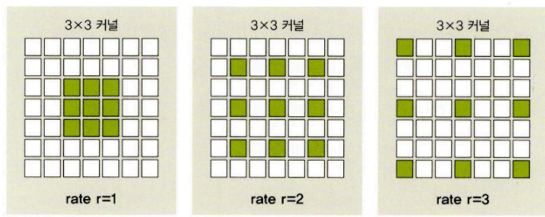
DeepLabv3 / DeepLabv3+

완전 연결층의 단점을 보완하기 위해 Atrous 합성곱을 사용한 네트워크

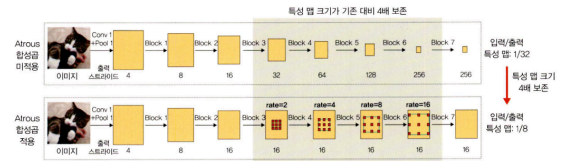


인코더와 디코더 구조를 가짐

- 이전에는 불가능했던 인코더에서 추출된 특성 맵의 해상도를 Atrous 합성곱을 도입하여 제어 가능하게 함



Atrous 합성곱



Atrous 합성곱 효과

Atrous 합성곱은 필터 내부에 빈 공간을 둔 채로 작동

파라미터 rate에 따라 얼마나 많은 빈공간을 가질지 결정

이미지 분할에서 높은 성능을 내려면 수용 영역의 크기가 중요

수용 영역을 확대하여 특성을 찾는 범위를 넓게 해줌

Atrous 합성곱을 활용하면 파라미터 수를 늘리지 않으면서도 수용 영역을 크게 키울 수 있음